Unidad 9 · SQL para consultar una tabla

Qué vas a poder hacer. Traducir una pregunta en palabras a un `SELECT` sobre una tabla: elegir columnas y calcular otras, filtrar filas con condiciones que no se traguen los `NULL`, ordenar, cortar las primeras filas y clasificar con `CASE`; leer la consulta en el orden en que el motor la evalúa y saber qué cambia entre SQLite y MySQL.

Lo que necesitas antes. De la unidad 7, la selección (σ, quedarse con filas) y la proyección (π, quedarse con columnas). De la unidad 8, cómo se crea y se llena una tabla (`CREATE TABLE`, `INSERT`) y los tipos `INTEGER`, `VARCHAR` y `DATE`. Nada más: esta unidad trabaja con una sola tabla a la vez.

Tiempo. Unas 5 horas: 2 de lectura con SQLite abierto y 3 de práctica. Todo se corre con `sql/u09_sql_una_tabla.sql` (taller y CESFAM) y `sql/u09_caso.sql` (sobre `datos/estampa_sur.db`).

> **Cómo leer los resultados.** Cada consulta de esta unidad se ejecutó en SQLite 3.50 y su resultado aparece tal como lo entrega el motor: sin punto de miles y con punto decimal (`141015.0`). En la lectura, las cifras van en formato chileno ($141.015).

---

1. La idea: se describe el resultado, no el camino

Un `SELECT` dice **qué** filas y columnas quieres, no **cómo** buscarlas. El motor decide si recorre la tabla entera o usa un índice (unidad 12); tú solo describes el resultado. Y el resultado de una consulta es otra tabla: tiene columnas con nombre y filas, y se puede seguir consultando (unidad 11).

La traducción desde el álgebra de la unidad 7 es directa, con dos diferencias que importan:

Álgebra relacional	SQL	La diferencia
$\sigma_{\text{condición}}(R)$	`WHERE condición`	ninguna
$\pi_{a,b}(R)$	`SELECT a, b`	SQL **no** elimina filas repetidas; para eso está `DISTINCT`
no existe	`ORDER BY`	una relación no tiene orden; una consulta sin `ORDER BY` puede devolver las filas en cualquier orden

La segunda fila es la fuente de muchos errores: el álgebra trabaja con conjuntos y SQL con **multiconjuntos** (una misma fila puede aparecer varias veces).

La tabla de la unidad. Un taller mecánico de Los Ángeles guarda sus órdenes de trabajo en una tabla. `entrega` vale `NULL` mientras el auto sigue en el taller y `monto` (neto, sin IVA) vale `NULL` mientras falta el presupuesto:

```
SELECT * FROM orden_trabajo;
```

id_ot	patente	marca	servicio	ingreso	entrega	km	monto	detalle
1	EJAE42	Toyota	mantención	2026-09-01	2026-09-01	85200	64900	aceite y filtros
2	EJAB17	Chevrolet	frenos	2026-09-02	2026-09-03	132400	118500	pastillas y discos delanteros
3	AE5821	Nissan	embrague	2026-09-02	2026-09-05	210700	389000	kit de embrague
4	EJAI63	Hyundai	mantención	2026-09-04	2026-09-04	40100	72400	aceite, filtros y revisión
5	EJAK28	Kia	diagnóstico	2026-09-07	NULL	98300	NULL	ruido en la suspensión
6	EJAC90	Toyota	frenos	2026-09-08	2026-09-09	156900	86300	pastillas traseras
7	AE4417	Suzuki	alineación	2026-09-09	2026-09-09	187500	28000	alineación y balanceo
8	EJAM55	Mazda	mantención	2026-09-10	2026-09-11	61800	79900	aceite, filtros y bujías
9	EJAA31	Chevrolet	embrague	2026-09-14	NULL	176200	412000	embrague patina
10	EJAF74	Hyundai	frenos	2026-09-15	NULL	119600	97800	líquido y pastillas delanteras
11	AE7302	Nissan	diagnóstico	2026-09-16	NULL	243100	NULL	luz de motor encendida
12	EJAN16	Kia	alineación	2026-09-16	2026-09-16	72900	26000	alineación

`SELECT *` trae todas las columnas. Sirve para mirar una tabla; en una consulta que va a un informe o a un programa conviene nombrar las columnas, para que el resultado no cambie si mañana alguien agrega una.

2. La forma de un `SELECT` y el orden en que se evalúa

Una consulta sobre una tabla tiene hasta cinco cláusulas, y se **escriben** siempre en este orden:

```
SELECT   columnas y expresiones
FROM     tabla
WHERE    condición sobre cada fila
ORDER BY criterio
LIMIT    cuántas filas
```

Pero el motor las **evalúa** en otro orden, y entenderlo explica casi todos los errores de la unidad:

Paso	Cláusula	Qué hace	Qué existe en ese momento
1	`FROM`	toma la tabla	todas las filas, todas las columnas
2	`WHERE`	se queda con las filas cuya condición es **verdadera**	las columnas de la tabla (los alias todavía no existen)
3	`SELECT`	calcula las columnas pedidas y les pone nombre	aparecen los alias
4	`DISTINCT`	elimina las filas repetidas del resultado	
5	`ORDER BY`	ordena	se pueden usar los alias
6	`LIMIT`	corta	

Dos consecuencias prácticas:

- **Un alias definido en el `SELECT` no existe en el `WHERE`** (paso 2 antes que el 3). En el estándar y en MySQL, filtrar por un alias da error; SQLite lo tolera (sección 3).
- **`LIMIT` corta después de ordenar.** «Los tres más caros» es `ORDER BY ... DESC LIMIT 3`; sin el `ORDER BY`, son tres filas cualquiera.

La unidad 10 agrega `GROUP BY` y `HAVING` entre los pasos 2 y 3.

---

3. Columnas: proyección, expresiones, alias y `DISTINCT`

Proyección. Se nombran las columnas en el orden en que se quieren ver:

```
SELECT patente, servicio, monto
FROM orden_trabajo
WHERE servicio = 'frenos';
```

patente	servicio	monto
EJAB17	frenos	118500
EJAC90	frenos	86300
EJAF74	frenos	97800

Expresiones y alias. Una columna del resultado puede ser un cálculo. `AS` le pone nombre; sin alias, la columna se llama como la expresión (`ROUND(monto * 1.19)`), incómodo de leer y de usar desde un programa.

```
SELECT patente, monto, ROUND(monto * 1.19) AS con_iva
FROM orden_trabajo
WHERE servicio = 'frenos';
```

patente	monto	con_iva
EJAB17	118500	141015.0
EJAC90	86300	102697.0
EJAF74	97800	116382.0

En SQLite `ROUND` devuelve un número real (`141015.0`); MySQL devuelve `141015`. El valor es el mismo.

El alias en el `WHERE`. Por el orden de evaluación, esto no debería funcionar:

```
SELECT patente, ROUND(monto * 1.19) AS con_iva
FROM orden_trabajo
WHERE con_iva > 140000;
```

patente	con_iva
EJAB17	141015.0
AE5821	462910.0
EJAA31	490280.0

SQLite lo acepta: si un nombre del `WHERE` no es una columna, lo busca entre los alias. Es una extensión propia. MySQL responde `Unknown column 'con_iva' in 'where clause'` (error 1054), y lo mismo PostgreSQL o SQL Server. La forma que corre en todos repite la expresión:

```
SELECT patente, ROUND(monto * 1.19) AS con_iva
FROM orden_trabajo
WHERE monto * 1.19 > 140000;
```

patente	con_iva
EJAB17	141015.0
AE5821	462910.0
EJAA31	490280.0

`DISTINCT`. ¿Qué marcas tienen autos con más de 150.000 km? Sin `DISTINCT`, Nissan sale dos veces, porque hay dos Nissan:

```
SELECT marca FROM orden_trabajo WHERE km > 150000;
```

marca
Nissan
Toyota
Suzuki
Chevrolet
Nissan

```
SELECT DISTINCT marca FROM orden_trabajo WHERE km > 150000 ORDER BY marca;
```

marca
Chevrolet
Nissan
Suzuki
Toyota

`DISTINCT` se aplica a la fila completa del resultado, no a una columna: `SELECT DISTINCT marca, servicio` elimina las combinaciones repetidas de marca y servicio.

---

4. Filas: `WHERE`

`WHERE` evalúa la condición **fila por fila** y se queda con las filas en que es verdadera. Las que dan falso o desconocido (sección 4.3) quedan fuera.

4.1 Comparaciones y cómo se escriben los datos

Los operadores son `=`, `<>` (distinto; `!=` también funciona en SQLite y MySQL), `<`, `<=`, `>` y `>=`. Lo que falla casi siempre no es el operador sino el dato escrito al lado:

Dato	Se escribe	Por qué
texto	`'frenos'`, entre comillas **simples**	sin comillas, `frenos` es el nombre de una columna; las comillas dobles son para nombres de columnas en el estándar
fecha	`'2026-09-07'`, texto en formato año-mes-día	así lo guarda SQLite y lo acepta MySQL; en ese formato el orden alfabético coincide con el cronológico
número	`140000`, sin punto de miles	`140.000` es el número 140 con tres decimales

Las comillas «curvas» (“así”) que pone un procesador de texto no son comillas para SQL: copia la consulta desde un editor de código.

4.2 `BETWEEN`, `IN` y `LIKE`

`BETWEEN a AND b` incluye los dos extremos. Los autos que ingresaron entre el 7 y el 9 de septiembre:

```
SELECT id_ot, patente, ingreso
FROM orden_trabajo
WHERE ingreso BETWEEN '2026-09-07' AND '2026-09-09';
```

id_ot	patente	ingreso
5	EJAK28	2026-09-07
6	EJAC90	2026-09-08
7	AE4417	2026-09-09

`IN (lista)` reemplaza una cadena de `OR` sobre la misma columna:

```
SELECT id_ot, patente, servicio
FROM orden_trabajo
WHERE servicio IN ('embrague', 'alineación');
```

id_ot	patente	servicio
3	AE5821	embrague
7	AE4417	alineación
9	EJAA31	embrague
12	EJAN16	alineación

`LIKE` compara texto con un patrón. `%` es «cualquier texto, incluso vacío» y `_` es «exactamente un carácter».

Patrón	Calza con	No calza con
`'pastillas%'`	«pastillas traseras»	«líquido y pastillas delanteras»
`'%pastillas%'`	las dos anteriores	«kit de embrague»
`'%delanteras'`	«líquido y pastillas delanteras»	«pastillas y discos delanteros» (termina en «os»)
`'____42'`	«EJAE42» (cuatro caracteres y luego 42)	«AE7302»

```
SELECT id_ot, patente, detalle
FROM orden_trabajo
WHERE detalle LIKE '%pastillas%';
```

id_ot	patente	detalle
2	EJAB17	pastillas y discos delanteros
6	EJAC90	pastillas traseras
10	EJAF74	líquido y pastillas delanteras

Un `LIKE` sin `%` ni `_` es una igualdad: `LIKE 'pastillas'` solo calza con el texto «pastillas» exacto. **Mayúsculas:** en SQLite, `LIKE` no distingue mayúsculas en las letras sin tilde (`'A' LIKE 'a'` es verdadero) pero sí en las demás (`'Ñ' LIKE 'ñ'` es falso). En MySQL depende de la intercalación (*collation*) de la columna; la de omisión de MySQL 8 no distingue mayúsculas ni tildes.

4.3 `NULL` y la lógica de tres valores

`NULL` no es un valor: es la marca de «no se sabe» o «no aplica». Por eso cualquier comparación con `NULL` da **desconocido**, ni verdadero ni falso. ¿Es la fecha de entrega de EJAK28 igual a `NULL`? No se sabe, porque no se sabe la fecha. Y como `WHERE` solo deja pasar lo verdadero:

```
SELECT id_ot, patente FROM orden_trabajo WHERE entrega = NULL;
```

La consulta devuelve 0 filas (columnas `id_ot`, `patente`).

No hay error, no hay aviso: cero filas. Para preguntar por la marca se usa `IS NULL` (o `IS NOT NULL`), que siempre responde verdadero o falso:

```
SELECT id_ot, patente, servicio FROM orden_trabajo WHERE entrega IS NULL;
```

id_ot	patente	servicio
5	EJAK28	diagnóstico
9	EJAA31	embrague
10	EJAF74	frenos
11	AE7302	diagnóstico

`NULL` se contagia. Una cuenta con un `NULL` da `NULL`. Si se quiere un valor de reemplazo, `COALESCE(x, valor)` devuelve el primer argumento que no sea `NULL` (en MySQL también `IFNULL`):

```
SELECT id_ot, monto, monto * 1.19 AS con_iva, COALESCE(monto, 0) AS monto_o_cero
FROM orden_trabajo
WHERE servicio = 'diagnóstico';
```

id_ot	monto	con_iva	monto_o_cero
5	NULL	NULL	0
11	NULL	NULL	0

Ojo: reemplazar un monto desconocido por 0 es una decisión de negocio, no un truco técnico. «Sin presupuesto» no es «gratis».

Las tablas de verdad con desconocido (V verdadero, F falso, D desconocido):

p	q	p AND q	p OR q	NOT p
V	D	D	V	F
F	D	F	D	V
D	D	D	D	D

La intuición: si p es falso, `p AND q` es falso sin importar q; si p es verdadero, `p OR q` es verdadero sin importar q. En los demás casos manda lo que no se sabe.

4.4 `AND`, `OR`, `NOT` y los paréntesis

`AND` se evalúa **antes** que `OR`, como la multiplicación antes que la suma. Se quieren los trabajos de frenos o de embrague que siguen en el taller. Escrito sin paréntesis:

```
SELECT id_ot, servicio, entrega
FROM orden_trabajo
WHERE servicio = 'frenos' OR servicio = 'embrague' AND entrega IS NULL;
```

id_ot	servicio	entrega
2	frenos	2026-09-03
6	frenos	2026-09-09
9	embrague	NULL
10	frenos	NULL

El motor leyó «frenos, **o bien** embrague sin entregar»: los frenos ya entregados (2 y 6) se colaron. Con paréntesis:

```
SELECT id_ot, servicio, entrega
FROM orden_trabajo
WHERE (servicio = 'frenos' OR servicio = 'embrague') AND entrega IS NULL;
```

id_ot	servicio	entrega
9	embrague	NULL
10	frenos	NULL

Regla: **cada vez que un `WHERE` mezcla `AND` y `OR`, se ponen paréntesis**, aunque la precedencia dé lo correcto. Quien lea la consulta después no tiene que adivinar.

---

5. Ordenar y cortar: `ORDER BY` y `LIMIT`

`ORDER BY` acepta varias claves: ordena por la primera y, dentro de cada empate, por la segunda. `ASC` (ascendente) es el orden de omisión; `DESC` invierte cada clave por separado.

```
SELECT servicio, patente, monto
FROM orden_trabajo
WHERE monto IS NOT NULL
ORDER BY servicio, monto DESC;
```

servicio	patente	monto
alineación	AE4417	28000
alineación	EJAN16	26000
embrague	EJAA31	412000
embrague	AE5821	389000
frenos	EJAB17	118500
frenos	EJAF74	97800
frenos	EJAC90	86300
mantención	EJAM55	79900
mantención	EJAI63	72400
mantención	EJAE42	64900

Los `NULL` van primero en orden ascendente, en SQLite y en MySQL (por eso se filtraron aquí).

`LIMIT n` deja las primeras n filas y `OFFSET k` se salta las primeras k. Los tres autos con más kilómetros y los tres siguientes, como una lista en páginas:

```
SELECT patente, km FROM orden_trabajo ORDER BY km DESC LIMIT 3;
```

patente	km
AE7302	243100
AE5821	210700
AE4417	187500

```
SELECT patente, km FROM orden_trabajo ORDER BY km DESC LIMIT 3 OFFSET 3;
```

patente	km
EJAA31	176200
EJAC90	156900
EJAB17	132400

`LIMIT` no es estándar. SQLite, MySQL y PostgreSQL lo entienden. El estándar SQL escribe `FETCH FIRST 3 ROWS ONLY` (con `OFFSET 3 ROWS` delante para saltar), que usan PostgreSQL y Oracle desde la versión 12c; SQL Server usa `SELECT TOP 3 ...` o `OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY`. SQLite **no** acepta `FETCH FIRST`. MySQL acepta además `LIMIT 3, 3` (primero el salto, después la cantidad), una forma fácil de leer al revés.

Empates en el corte. Si hay empate en la última posición, cuál fila entra es arbitrario. Para que la consulta dé siempre lo mismo, agrega una clave de desempate única (la clave primaria) al final del `ORDER BY`. El ejercicio 9.7 muestra un empate de cinco filas.

---

6. Funciones de número, texto y fecha

Las funciones varían más entre motores que cualquier otra parte de SQL. Esta sección muestra las de SQLite, que corren en esta base, y su equivalente en MySQL.

Números: la división entera. En SQLite, dividir dos enteros da un entero: se pierden los decimales sin aviso. Basta con que uno de los dos sea real (`1000.0`):

```
SELECT patente, km / 1000 AS miles_entera, km / 1000.0 AS miles, ROUND(km / 1000.0, 1) AS
miles_1
FROM orden_trabajo
WHERE id_ot <= 3;
```

patente	miles_entera	miles	miles_1
EJAE42	85	85.2	85.2
EJAB17	132	132.4	132.4
AE5821	210	210.7	210.7

En MySQL, `/` siempre da decimales (`85200 / 1000` es `85.2000`) y la división entera se pide con `DIV`. Es la diferencia más traicionera entre los dos motores. Con enteros, en SQLite, `100 * 1170 / 3190` da 36 en vez de 36,7, y `1170 / 3190 * 100` da 0. Escribe `100.0 * a / b`.

Texto.

```
SELECT marca || ' ' || patente AS auto, UPPER(servicio) AS servicio_may,
       LENGTH(detalle) AS largo, SUBSTR(patente, 1, 2) AS inicio
FROM orden_trabajo
WHERE id_ot IN (2, 7, 11);
```

auto	servicio_may	largo	inicio
Chevrolet EJAB17	FRENOS	29	EJ
Suzuki AE4417	ALINEACIóN	21	AE
Nissan AE7302	DIAGNóSTICO	22	AE

Mira la segunda columna: el `UPPER` de SQLite solo convierte letras sin tilde («ALINEACIóN»). No es un error de la tabla: SQLite, sin extensiones, no conoce las reglas de mayúsculas fuera del inglés.

Fechas. SQLite guarda las fechas como texto `'AAAA-MM-DD'` y las trata con funciones:

```
SELECT patente, strftime('%d-%m-%Y', ingreso) AS ingreso_cl,
       julianday(entrega) - julianday(ingreso) AS dias,
       date(ingreso, '+30 days') AS garantia_hasta
FROM orden_trabajo
WHERE servicio = 'embrague';
```

patente	ingreso_cl	dias	garantia_hasta
AE5821	02-09-2026	3.0	2026-10-02
EJAA31	14-09-2026	NULL	2026-10-14

`julianday` convierte una fecha en un número de días; la resta da los días entre dos fechas. EJAA31 no tiene entrega, y el `NULL` se contagió a la resta.

La tabla de equivalencias.

Para	SQLite	MySQL
concatenar	`a || b` (y `concat(a, b)` desde la versión 3.44)	`CONCAT(a, b)`; `||` es un `OR` lógico salvo que se active `PIPES_AS_CONCAT`
largo de un texto	`LENGTH(x)`, en caracteres	`CHAR_LENGTH(x)`; `LENGTH(x)` cuenta bytes («alineación y balanceo» mide 21 caracteres y 22 bytes)
mayúsculas	`UPPER(x)`, solo letras sin tilde	`UPPER(x)`, también con tildes y eñes
subtexto	`SUBSTR(x, desde, largo)`	`SUBSTRING(x, desde, largo)` (también `SUBSTR`)
posición de un texto	`INSTR(x, ' ')`	`INSTR(x, ' ')` o `LOCATE(' ', x)`
quitar espacios, reemplazar	`TRIM(x)`, `REPLACE(x, 'a', 'b')`	iguales
redondear, valor absoluto	`ROUND(x, d)` (devuelve real), `ABS(x)`	`ROUND(x, d)`, `ABS(x)`
división entera	`7 / 2` da 3	`7 DIV 2` da 3; `7 / 2` da 3.5000
formatear una fecha	`strftime('%d-%m-%Y', f)`	`DATE_FORMAT(f, '%d-%m-%Y')`
mes y año	`strftime('%m', f)` (texto `'09'`)	`MONTH(f)`, `YEAR(f)` (números)
días entre dos fechas	`julianday(b) - julianday(a)` (real)	`DATEDIFF(b, a)` (entero)
sumar días	`date(f, '+30 days')`	`DATE_ADD(f, INTERVAL 30 DAY)`
fecha de hoy	`date('now')`	`CURDATE()`
reemplazar un `NULL`	`COALESCE(x, v)`, `IFNULL(x, v)`	`COALESCE(x, v)`, `IFNULL(x, v)`

`COALESCE`, `CASE`, `SUBSTRING`, `CHAR_LENGTH`, `UPPER`, `TRIM` y `||` son del estándar; el resto depende del motor. Cuando una consulta tiene que correr en los dos, prefiere las del estándar.

---

7. `CASE`: clasificar fila por fila

`CASE` es el «si… entonces…» de SQL. Evalúa las condiciones `WHEN` en orden y devuelve el `THEN` de la **primera** que es verdadera; si ninguna lo es, el `ELSE` (y si no hay `ELSE`, `NULL`).

```
SELECT id_ot, monto,
       CASE
           WHEN monto IS NULL THEN 'sin presupuesto'
           WHEN monto < 50000 THEN 'menor'
           WHEN monto < 150000 THEN 'medio'
           ELSE 'mayor'
       END AS tramo
FROM orden_trabajo
ORDER BY id_ot;
```

id_ot	monto	tramo
1	64900	medio
2	118500	medio
3	389000	mayor
4	72400	medio
5	NULL	sin presupuesto
6	86300	medio
7	28000	menor
8	79900	medio
9	412000	mayor
10	97800	medio
11	NULL	sin presupuesto
12	26000	menor

Por qué funciona sin escribir `monto >= 50000 AND monto < 150000`: si una fila llega al tercer `WHEN`, ya se sabe que no cumplió el segundo. **El orden de los `WHEN` importa**, y el `NULL` va primero: `NULL < 50000` es desconocido, y sin la primera línea los montos sin presupuesto caerían en «mayor» por el `ELSE`.

---

8. Contar filas (adelanto)

`COUNT(*)` cuenta las filas que quedan después del `WHERE`. Es una función de agregación: junta muchas filas en una. La unidad 10 las trata en serio; acá basta con contar.

```
SELECT COUNT(*) AS en_taller FROM orden_trabajo WHERE entrega IS NULL;
```

en_taller
4

---

9. Cuando el texto del `WHERE` lo escribe otra persona

Un programa que busca a un paciente por su nombre tiende a armar la consulta **pegando** el texto que escribe el usuario dentro del `WHERE`. Si ese texto trae una comilla, la comilla cierra el texto de la consulta y lo que viene después pasa a ser SQL: es la **inyección SQL**, uno de los ataques más viejos y todavía frecuentes. El ejercicio 9.10 lo muestra corriendo.

La defensa no es «limpiar» el texto, sino no pegarlo nunca: la consulta lleva un marcador (`?` en el módulo `sqlite3` de Python, `%s` en el conector de MySQL) y el texto viaja aparte, como un dato. El motor nunca lo lee como SQL. La unidad 12 vuelve sobre esto al usar la base desde un programa.

10. Ejemplo resuelto · El taller el jueves 17-09-2026

El dueño llega temprano y tiene tres preguntas. Cada consulta se lee en el orden en que el motor la evalúa.

Pregunta 1 · ¿Qué autos siguen aquí y hace cuántos días?

```
SELECT patente, servicio, ingreso,
       julianday('2026-09-17') - julianday(ingreso) AS dias_en_taller
FROM orden_trabajo
WHERE entrega IS NULL
ORDER BY dias_en_taller DESC, patente;
```

- `FROM`: las 12 órdenes.
- `WHERE entrega IS NULL`: quedan 4 (con `= NULL` no quedaría ninguna).
- `SELECT`: calcula los días desde el ingreso hasta el 17-09-2026 y los llama `dias_en_taller`.
- `ORDER BY`: el más antiguo arriba; la patente desempata.

patente	servicio	ingreso	dias_en_taller
EJAK28	diagnóstico	2026-09-07	10.0
EJAA31	embrague	2026-09-14	3.0
EJAF74	frenos	2026-09-15	2.0
AE7302	diagnóstico	2026-09-16	1.0

Pregunta 2 · ¿Cuánto se facturó, con IVA, en frenos y embragues ya entregados? La suma de varias filas es tema de la unidad 10; por ahora basta con la lista ordenada por total.

```
SELECT patente, servicio, monto, ROUND(monto * 1.19) AS total_con_iva
FROM orden_trabajo
WHERE entrega IS NOT NULL
  AND servicio IN ('frenos', 'embrague')
ORDER BY total_con_iva DESC;
```

patente	servicio	monto	total_con_iva
AE5821	embrague	389000	462910.0
EJAB17	frenos	118500	141015.0
EJAC90	frenos	86300	102697.0

Aquí `AND` e `IN` no necesitan paréntesis: no hay `OR` suelto. Escrito con `OR` sí harían falta: `entrega IS NOT NULL AND (servicio = 'frenos' OR servicio = 'embrague')`.

Pregunta 3 · ¿Cumplimos los plazos de entrega?

```
SELECT patente, servicio,
       julianday(entrega) - julianday(ingreso) AS dias,
       CASE
           WHEN entrega = ingreso THEN 'mismo día'
           WHEN julianday(entrega) - julianday(ingreso) <= 1 THEN 'al día siguiente'
           ELSE 'más de un día'
       END AS plazo
FROM orden_trabajo
WHERE entrega IS NOT NULL
ORDER BY dias DESC, patente;
```

patente	servicio	dias	plazo
AE5821	embrague	3.0	más de un día
EJAB17	frenos	1.0	al día siguiente
EJAC90	frenos	1.0	al día siguiente
EJAM55	mantención	1.0	al día siguiente
AE4417	alineación	0.0	mismo día
EJAE42	mantención	0.0	mismo día
EJAI63	mantención	0.0	mismo día
EJAN16	alineación	0.0	mismo día

Cómo se lee.

- **Lo urgente es EJAK28.** Lleva 10 días en diagnóstico, sin presupuesto (`monto` en `NULL`): el cliente no sabe cuánto va a pagar. Hay que cerrar ese diagnóstico antes de recibir otro.
- **Frenos y embragues entregados suman $706.622 con IVA** (462.910 + 141.015 + 102.697). El embrague de AE5821 es el 66 % de eso: un solo trabajo grande pesa más que varios chicos.
- **Los plazos se cumplen.** De los 8 entregados, 4 salieron el mismo día y 3 al día siguiente; solo el embrague de AE5821 tardó 3 días, razonable para un kit completo. El problema del taller no es la velocidad de los trabajos, sino el diagnóstico que quedó detenido.

11. Errores típicos

Error	Cómo evitarlo
`WHERE x = NULL` (o `<> NULL`): cero filas, sin aviso	`IS NULL` e `IS NOT NULL`; ninguna comparación con `NULL` es verdadera
Mezclar `AND` y `OR` sin paréntesis	`AND` va antes que `OR`; pon paréntesis siempre que aparezcan los dos
Texto sin comillas (`profesional = medicina`)	sin comillas es una columna: el motor responde `no such column`
Comillas dobles o curvas para el texto	comillas simples rectas; en SQLite, `"x"` es una columna si existe una con ese nombre
Fechas como `07/10/2026` o `'07/10/2026'`	sin comillas es una división (da 0); entre comillas, `'08/10/2026' > '07/11/2026'` se compara como texto; usa `'2026-10-07'`
Punto de miles en un número (`precio > 5.000`)	es el número 5; escribe `5000`
`LIKE` sin `%` (`LIKE 'Muñoz'`)	sin comodines es una igualdad; `'%Muñoz'` para «termina en»
Usar en el `WHERE` un alias del `SELECT`	el `WHERE` va antes que el `SELECT`; repite la expresión (SQLite lo tolera, MySQL no)
`LIMIT` sin `ORDER BY`, o con empates	sin orden, el corte es arbitrario; agrega la clave primaria como desempate
División entera en SQLite (`100 * a / b`)	escribe `100.0 * a / b`
Pegar el texto de un usuario dentro de la consulta	parámetros (`?`), nunca concatenación

12. Práctica

Diez ejercicios: 9.1 a 9.4 básicos, 9.5 a 9.8 intermedios, 9.9 y 9.10 de desafío. Los ejercicios 9.1 a 9.3, 9.5, 9.6, 9.8 a 9.10 usan la tabla `hora` de un CESFAM de Coronel, que crea `sql/u09_sql_una_tabla.sql`; los ejercicios 9.4 y 9.7 y la parte (e) del 9.9 usan la base del caso. Escribe cada consulta y córrela antes de mirar la pauta.

La tabla `hora` (una semana de horas agendadas; hoy es miércoles 07-10-2026). `asistio` vale 1 si el paciente llegó, 0 si faltó y `NULL` si la hora todavía no llega; `espera_min` son los minutos en sala de espera y vale `NULL` si no hubo atención.

id_hora	paciente	edad	sector	profesional	fecha	asistio	espera_min
1	Rosa Contreras	71	azul	medicina	2026-10-05	1	35
2	Matías Fuentes	8	rojo	dental	2026-10-05	1	12
3	Javiera Muñoz	29	verde	matrona	2026-10-05	0	NULL
4	Luis Sepúlveda	66	azul	enfermería	2026-10-06	1	48
5	Ana Riquelme	15	verde	dental	2026-10-06	1	20
6	Pedro Muñoz	80	rojo	medicina	2026-10-06	1	55
7	Camila Vergara	33	azul	matrona	2026-10-07	1	9
8	Jorge Toledo	58	verde	nutrición	2026-10-07	0	NULL
9	Sofía Mella	12	rojo	enfermería	2026-10-07	1	15
10	Héctor Muñoz	69	azul	medicina	2026-10-07	1	41
11	Valentina Soto	24	rojo	matrona	2026-10-08	NULL	NULL
12	Raúl Castillo	74	verde	medicina	2026-10-08	NULL	NULL
13	Daniela Pino	17	rojo	dental	2026-10-09	NULL	NULL
14	Manuel Ortiz	45	azul	nutrición	2026-10-09	NULL	NULL

9.1 · Adultos mayores del sector azul (básico). El equipo del sector azul quiere llamar a sus pacientes de 65 años o más. Lista nombre y edad, del mayor al menor.

9.2 · Horas de matrona y dental (básico). Lista paciente, profesional y fecha de las horas de matrona o dental entre el lunes 05-10 y el miércoles 07-10, ordenadas por fecha y luego por paciente. Usa `IN` y `BETWEEN`.

9.3 · Lo que hay y lo que falta (básico). (a) ¿Qué profesionales distintos atienden esta semana? Ordénalos alfabéticamente. (b) ¿Cuántas horas no tienen asistencia registrada? (c) Un compañero escribió `WHERE asistio = NULL` para la parte (b). ¿Qué número obtiene y por qué?

9.4 · Productos de precio medio en Estampa Sur (básico, caso). En la tabla `producto` de la base del caso, lista nombre, precio, margen unitario (`precio - costo_var`, con alias `margen`) y margen porcentual con un decimal, de los productos con precio entre $3.000 y $7.000, ordenados por margen de mayor a menor. ¿El orden sería el mismo si se ordenara por margen porcentual?

9.5 · Una condición que dice otra cosa (intermedio). El programa de salud escolar pide los menores de 18 años de los sectores rojo y verde. Alguien escribió:

```
SELECT paciente, sector, edad
FROM hora
WHERE sector = 'rojo' OR sector = 'verde' AND edad < 18
ORDER BY id_hora;
```

(a) Sin correrla, di qué filas devuelve y por qué. (b) Corrígela. (c) ¿Cuántas filas de más traía la versión con error?

9.6 · Tramos de espera (intermedio). Para las horas del lunes al miércoles (hasta el 07-10), clasifica la espera en «a tiempo» (hasta 15 minutos), «demora» (16 a 40), «larga» (más de 40) y «sin dato» (cuando no hubo atención). Muestra paciente, minutos y tramo, en el orden de las horas. ¿Qué pasa si pones el `WHEN espera_min IS NULL` al final, antes del `ELSE`?

9.7 · Pedidos abiertos de la segunda quincena (intermedio, caso). En la tabla `pedido` de Estampa Sur: (a) ¿cuántos pedidos de entre el 16 y el 31 de marzo están pendientes o en taller? (b) Lista los tres más antiguos (número, fecha y estado). (c) ¿Por qué la pregunta (b) está mal planteada? Cuéntalo con una consulta.

9.8 · El recordatorio (intermedio). El CESFAM manda un recordatorio por cada hora que todavía no llega. Para cada una, muestra el apellido (todo lo que sigue al primer espacio del nombre), el día en formato DD-MM-AAAA, cuántos días faltan contados desde el 07-10-2026 y el profesional, ordenado por fecha y apellido. Después escribe en una nota qué funciones cambiarías para correrlo en MySQL.

9.9 · Cinco consultas que no dicen lo que parecen (desafío). Un practicante escribió estas consultas. Para cada una, di qué devuelve SQLite y por qué, y corrígela.

```
SELECT paciente FROM hora WHERE profesional = medicina;
```

```
SELECT paciente FROM hora WHERE espera_min = NULL;
```

```
SELECT paciente FROM hora WHERE fecha = 07/10/2026;
```

```
SELECT paciente FROM hora WHERE paciente LIKE 'Muñoz';
```

```
SELECT nombre, precio FROM producto WHERE precio > 5.000;
```

(Las cuatro primeras son sobre `hora`: la (a) busca las horas de medicina; la (b), las horas sin atención; la (c), las del 07-10-2026; la (d), los pacientes de apellido Muñoz. La (e) es sobre la base del caso y busca los productos de más de $5.000.)

9.10 · El buscador del mesón (desafío). El mesón del CESFAM tiene un pequeño programa en Python para buscar a un paciente por su nombre:

```
import sqlite3
con = sqlite3.connect("u09.db")
texto = input("Paciente: ")
sql = "SELECT paciente, edad, profesional FROM hora WHERE paciente = '" + texto + "'"
filas = con.execute(sql).fetchall()
```

(a) ¿Qué consulta llega al motor si alguien escribe `Rosa Contreras`? ¿Y si escribe `x' OR '1'='1`? (b) ¿Cuántas filas devuelve cada una? (c) ¿Por qué esto es un problema bajo la ley de datos personales (Ley 21.719), aunque nadie haya «hackeado» nada? (d) Corrige el programa.

13. Pautas

9.1.

```
SELECT paciente, edad
FROM hora
WHERE sector = 'azul' AND edad >= 65
ORDER BY edad DESC;
```

paciente	edad
Rosa Contreras	71
Héctor Muñoz	69
Luis Sepúlveda	66

Tres llamadas. `>=` y no `>`: «65 o más» incluye a quien tiene 65.

9.2.

```
SELECT paciente, profesional, fecha
FROM hora
WHERE profesional IN ('matrona', 'dental')
  AND fecha BETWEEN '2026-10-05' AND '2026-10-07'
ORDER BY fecha, paciente;
```

paciente	profesional	fecha
Javiera Muñoz	matrona	2026-10-05
Matías Fuentes	dental	2026-10-05
Ana Riquelme	dental	2026-10-06
Camila Vergara	matrona	2026-10-07

`BETWEEN` incluye el 07-10 (Camila Vergara). Javiera aparece aunque faltó: la pregunta era por horas agendadas, no por atenciones.

9.3. (a)

```
SELECT DISTINCT profesional FROM hora ORDER BY profesional;
```

profesional
dental
enfermería
matrona
medicina
nutrición

(b)

```
SELECT COUNT(*) AS sin_registro FROM hora WHERE asistio IS NULL;
```

sin_registro
4

(c)

```
SELECT COUNT(*) AS sin_registro FROM hora WHERE asistio = NULL;
```

sin_registro
0

`asistio = NULL` es desconocido en todas las filas, y el `WHERE` no deja pasar ninguna. Lo peligroso es que 0 parece una respuesta razonable («todo registrado») y no lo es.

9.4.

```
SELECT nombre, precio, precio - costo_var AS margen,
       ROUND(100.0 * (precio - costo_var) / precio, 1) AS margen_pct
FROM producto
WHERE precio BETWEEN 3000 AND 7000
ORDER BY margen DESC;
```

nombre	precio	margen	margen_pct
poleras estampado doble	6290	2270	36.1
poleras básicas	3990	1380	34.6
gorros	3190	1170	36.7

No es el mismo orden. En pesos gana la polera de estampado doble ($2.270 por unidad); en porcentaje ganan los gorros (36,7 %). Cuál importa depende de qué limita: si limita la venta, el margen en pesos por unidad; la unidad de Optimización mostró que si limita una máquina, importa el margen por minuto de esa máquina. El `100.0` evita la división entera.

9.5. (a) `AND` se evalúa primero, así que la condición es `sector = 'rojo' OR (sector = 'verde' AND edad < 18)`: todos los del sector rojo, de cualquier edad, más los menores del verde.

```
SELECT paciente, sector, edad
FROM hora
WHERE sector = 'rojo' OR sector = 'verde' AND edad < 18
ORDER BY id_hora;
```

paciente	sector	edad
Matías Fuentes	rojo	8
Ana Riquelme	verde	15
Pedro Muñoz	rojo	80
Sofía Mella	rojo	12
Valentina Soto	rojo	24
Daniela Pino	rojo	17

(b)

```
SELECT paciente, sector, edad
FROM hora
WHERE (sector = 'rojo' OR sector = 'verde') AND edad < 18
ORDER BY id_hora;
```

paciente	sector	edad
Matías Fuentes	rojo	8
Ana Riquelme	verde	15
Sofía Mella	rojo	12
Daniela Pino	rojo	17

(c) Dos de más: Pedro Muñoz (80 años) y Valentina Soto (24). También se puede escribir `sector IN ('rojo', 'verde') AND edad < 18`, que no necesita paréntesis.

9.6.

```
SELECT paciente, espera_min,
       CASE
           WHEN espera_min IS NULL THEN 'sin dato'
           WHEN espera_min <= 15 THEN 'a tiempo'
           WHEN espera_min <= 40 THEN 'demora'
           ELSE 'larga'
       END AS tramo
FROM hora
WHERE fecha <= '2026-10-07'
ORDER BY id_hora;
```

paciente	espera_min	tramo
Rosa Contreras	35	demora
Matías Fuentes	12	a tiempo
Javiera Muñoz	NULL	sin dato
Luis Sepúlveda	48	larga
Ana Riquelme	20	demora
Pedro Muñoz	55	larga
Camila Vergara	9	a tiempo
Jorge Toledo	NULL	sin dato
Sofía Mella	15	a tiempo
Héctor Muñoz	41	larga

Tres esperas largas, las tres de pacientes de 66 años o más (medicina y enfermería). Si el `WHEN ... IS NULL` va al final, nada cambia en este caso: los `NULL` no cumplen ninguna comparación y llegan a ese `WHEN` antes del `ELSE`. Lo que sí falla es **olvidarlo**: entonces los `NULL` caen en el `ELSE` y salen como «larga». Ponerlo primero deja claro que se pensó.

9.7. (a)

```
SELECT COUNT(*) AS abiertos
FROM pedido
WHERE fecha BETWEEN '2026-03-16' AND '2026-03-31'
  AND estado IN ('pendiente', 'en taller');
```

abiertos
23

(b)

```
SELECT id_pedido, fecha, estado
FROM pedido
WHERE fecha BETWEEN '2026-03-16' AND '2026-03-31'
  AND estado IN ('pendiente', 'en taller')
ORDER BY fecha, id_pedido
LIMIT 3;
```

id_pedido	fecha	estado
60	2026-03-17	en taller
72	2026-03-17	pendiente
85	2026-03-17	en taller

(c) Porque los tres «más antiguos» empatan con otros:

```
SELECT COUNT(*) AS empatados
FROM pedido
WHERE fecha = '2026-03-17'
  AND estado IN ('pendiente', 'en taller');
```

empatados
5

Hay 5 pedidos del 17-03 abiertos, todos igual de antiguos. Con `ORDER BY fecha` solo, cuáles tres salen es arbitrario; con `id_pedido` como desempate la respuesta se repite, pero sigue siendo una elección, no un hecho. Lo honesto es informar los cinco. De los 23 pedidos abiertos, la fecha más antigua es el 17-03.

9.8.

```
SELECT SUBSTR(paciente, INSTR(paciente, ' ') + 1) AS apellido,
       strftime('%d-%m-%Y', fecha) AS dia,
       julianday(fecha) - julianday('2026-10-07') AS faltan,
       profesional
FROM hora
WHERE asistio IS NULL
ORDER BY fecha, apellido;
```

apellido	dia	faltan	profesional
Castillo	08-10-2026	1.0	medicina
Soto	08-10-2026	1.0	matrona
Ortiz	09-10-2026	2.0	nutrición
Pino	09-10-2026	2.0	dental

`INSTR` da la posición del primer espacio y `SUBSTR` desde la posición siguiente hasta el final (sin tercer argumento, toma el resto). El `ORDER BY` usa el alias `apellido`, que ya existe en ese paso. En MySQL: `SUBSTRING(paciente, LOCATE(' ', paciente) + 1)`, `DATE_FORMAT(fecha, '%d-%m-%Y')` y `DATEDIFF(fecha, '2026-10-07')`, que da 1 y 2 enteros. Ojo: un nombre con dos nombres de pila («María José Pérez») daría «José Pérez». El apellido debió ser una columna aparte (unidad 6: un dato por columna).

9.9. (a) El texto sin comillas es un nombre de columna:

```
SELECT paciente FROM hora WHERE profesional = medicina;
```

Error: no such column: medicina

```
SELECT paciente FROM hora WHERE profesional = 'medicina';
```

paciente
Rosa Contreras
Pedro Muñoz
Héctor Muñoz
Raúl Castillo

(b) `= NULL` nunca es verdadero: cero filas, sin error.

```
SELECT paciente FROM hora WHERE espera_min = NULL;
```

La consulta devuelve 0 filas (columnas `paciente`).

```
SELECT paciente FROM hora WHERE espera_min IS NULL;
```

paciente
Javiera Muñoz
Jorge Toledo
Valentina Soto
Raúl Castillo
Daniela Pino
Manuel Ortiz

Seis horas sin atención: dos faltas y cuatro horas que todavía no llegan. Si se quieren solo las faltas, es `asistio = 0`.

(c) Sin comillas, `07/10/2026` es una división de enteros: 7 dividido por 10 da 0, y 0 dividido por 2026 da 0.

```
SELECT paciente FROM hora WHERE fecha = 07/10/2026;
```

La consulta devuelve 0 filas (columnas `paciente`).

```
SELECT 07/10/2026 AS lo_que_compara;
```

lo_que_compara
0

```
SELECT paciente FROM hora WHERE fecha = '2026-10-07';
```

paciente
Camila Vergara
Jorge Toledo
Sofía Mella
Héctor Muñoz

Entre comillas pero en formato `'07/10/2026'` tampoco calza, porque la columna guarda `'2026-10-07'`; y comparar con `>` en ese formato ordena por día antes que por mes.

(d) Sin `%`, `LIKE` es una igualdad: ningún paciente se llama solo «Muñoz».

```
SELECT paciente FROM hora WHERE paciente LIKE 'Muñoz';
```

La consulta devuelve 0 filas (columnas `paciente`).

```
SELECT paciente FROM hora WHERE paciente LIKE '%Muñoz';
```

paciente
Javiera Muñoz
Pedro Muñoz
Héctor Muñoz

(e) `5.000` es el número 5 con tres decimales, así que salen todos los productos:

```
SELECT nombre, precio FROM producto WHERE precio > 5.000;
```

nombre	precio
poleras básicas	3990
polerones	9890
poleras estampado doble	6290
bolsas	2490
gorros	3190

```
SELECT nombre, precio FROM producto WHERE precio > 5000;
```

nombre	precio
polerones	9890
poleras estampado doble	6290

Lectura común a los cinco: solo el (a) da error. Los otros cuatro **corren** y devuelven algo creíble; por eso se revisa el resultado contra lo que uno espera (¿cuántas filas debería haber?) antes de usarlo.

9.10. (a) Con `Rosa Contreras` llega `... WHERE paciente = 'Rosa Contreras'`. Con `x' OR '1'='1` la comilla del usuario cierra el texto, y llega `... WHERE paciente = 'x' OR '1'='1'`: una condición que es verdadera en todas las filas. (b) La primera devuelve 1 fila; la segunda devuelve las 14 filas de la tabla, con nombre, edad y profesional de cada paciente. (c) Cualquier persona en el mesón puede ver la agenda completa con datos de salud de terceros, que son datos sensibles: el programa no protege la confidencialidad que la ley exige, aunque nadie haya entrado «desde afuera». (d) El texto viaja como parámetro y el motor lo compara como un dato, nunca lo lee como SQL:

```
sql = "SELECT paciente, edad, profesional FROM hora WHERE paciente = ?"
filas = con.execute(sql, (texto,)).fetchall()
```

Con `Rosa Contreras` sigue devolviendo 1 fila; con `x' OR '1'='1` devuelve 0 filas, porque nadie se llama así. Quitar las comillas del texto «a mano» no basta: hay otras formas de escapar del texto. La regla es simple: nunca se concatena lo que escribe el usuario.

---

14. Caso integrador · Estampa Sur pregunta por sus productos y pedidos

> El caso sigue a Estampa Sur por todo el ramo. La unidad 8 creó las 8 tablas y las llenó: 5 productos, 30 clientes, 120 pedidos de marzo de 2026 y 212 líneas de detalle. Esta unidad le hace las primeras preguntas, **una tabla a la vez**. La base está en `datos/estampa_sur.db` y las consultas en `sql/u09_caso.sql`.

¿Qué productos cuestan más de $4.000?

```
SELECT nombre, precio FROM producto WHERE precio > 4000 ORDER BY precio DESC;
```

nombre	precio
polerones	9890
poleras estampado doble	6290

¿Cuántos pedidos están pendientes?

```
SELECT COUNT(*) FROM pedido WHERE estado = 'pendiente';
```

COUNT(*)
8

¿Cuáles son y cuáles urgen? Al 01-04-2026, un pedido pendiente hace más de 14 días es urgente:

```
SELECT id_pedido, id_cliente, fecha,
       julianday('2026-04-01') - julianday(fecha) AS dias_esperando,
       CASE
           WHEN julianday('2026-04-01') - julianday(fecha) > 14 THEN 'urgente'
           ELSE 'normal'
       END AS prioridad
FROM pedido
WHERE estado = 'pendiente'
ORDER BY fecha, id_pedido;
```

id_pedido	id_cliente	fecha	dias_esperando	prioridad
68	17	2026-03-02	30.0	urgente
73	24	2026-03-08	24.0	urgente
92	24	2026-03-15	17.0	urgente
72	4	2026-03-17	15.0	urgente
15	14	2026-03-22	10.0	normal
28	14	2026-03-22	10.0	normal
32	10	2026-03-31	1.0	normal
120	22	2026-03-31	1.0	normal

¿Cuánto deja cada producto por unidad?

```
SELECT nombre, precio, costo_var,
       precio - costo_var AS margen,
       ROUND(100.0 * (precio - costo_var) / precio, 1) AS margen_pct
FROM producto
ORDER BY margen_pct DESC;
```

nombre	precio	costo_var	margen	margen_pct
polerones	9890	5950	3940	39.8
bolsas	2490	1530	960	38.6
gorros	3190	2020	1170	36.7
poleras estampado doble	6290	4020	2270	36.1
poleras básicas	3990	2610	1380	34.6

Lectura. Solo dos productos pasan de $4.000. De los 8 pedidos pendientes, 4 son urgentes, y el cliente 24 tiene dos de ellos (73 y 92): una llamada resuelve dos atrasos. Los márgenes porcentuales van de 34,6 % (poleras básicas) a 39,8 % (polerones): los precios del taller son parejos en porcentaje, así que lo que decide qué conviene vender es la capacidad, como mostraron Optimización y Gestión de Operaciones.

La cadena. El verificador corre `caso_estampa_sur_bd.py`, ejecuta `u09_caso.sql` sobre la base que deja y exige que las dos consultas `u9_*` sean las mismas del script del caso y den lo mismo (`CADENA: OK`).

Lo que pasa a la unidad siguiente. Hasta aquí cada pregunta cabía en una tabla. Pero «¿cuánto vendió cada tienda?» necesita el detalle (cantidades y precios), el pedido (de qué cliente es), el cliente (de qué tienda es) y la tienda (su nombre): cuatro tablas. Eso, y sumar por grupo, es la unidad 10.

---

15. Para seguir

En los libros del área (el programa del ramo no fija bibliografía):

- Elmasri y Navathe, *Fundamentos de sistemas de bases de datos*: el capítulo de SQL básico trae consultas sobre una tabla, `NULL` y la lógica de tres valores, con su base de ejemplo de una empresa.
- Silberschatz, Korth y Sudarshan, *Fundamentos de bases de datos*: el capítulo de introducción a SQL trata `SELECT`, `WHERE`, operaciones con texto y el orden de las filas.

Abiertos y gratis.

- SQLite, «SELECT» (en inglés): la sintaxis completa del `SELECT` en SQLite, con `LIMIT` y `OFFSET` y las rarezas propias del motor. https://www.sqlite.org/lang_select.html
- MySQL 8.4 Reference Manual, capítulo 14 «Functions and Operators» (en inglés): las funciones de texto, número y fecha de MySQL, para traducir las de la sección 6. https://dev.mysql.com/doc/refman/8.4/en/functions.html

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Dame una tabla de diez filas con algunos `NULL` y cinco preguntas en palabras; yo escribo el `SELECT` y tú me dices qué filas debería devolver antes de que lo corra».
